Chapter 3: Symbolic Regression

Discovering Physical Laws

Dr. Sam Smith

Outline

- Traditional regression vs. symbolic regression
- Cantilever beam deflection problem
- Genetic programming: expression trees, crossover, mutation
- Fitness and parsimony pressure
- Discovered equation vs. theory
- Generalization: training vs. test performance

Traditional Regression vs. Symbolic Regression

Traditional regression (Chapter 2):

- The model *form* is fixed in advance, example: $\hat{\sigma} = w_1 \varepsilon + w_0$.
- Training only adjusts numerical parameters w_0, w_1.

Symbolic regression (SR):

- Searches over both the *structure* (operators, variables) and the *constants* simultaneously.
- Result: an **interpretable, closed-form** equation.
- Contrast with **black-box** neural networks (Chapter 4).

The most established SR method is **Genetic Programming (GP)**, which evolves candidate expressions using operators inspired by biological evolution.

Problem Setup: Cantilever Beam Deflection

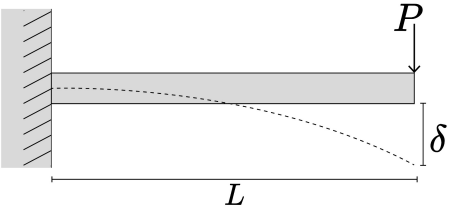
Predict the tip deflection δ of a cantilever beam under a point load P.

Exact solution (Euler–Bernoulli):

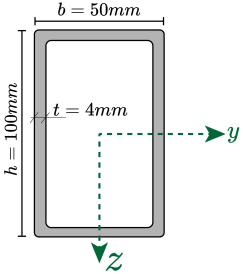
$$\delta = \frac{PL^3}{3EI}$$

We **pretend this formula is unknown** and let SR discover it from data.

- Steel RHS section ($E = 210$ GPa)
- $I = 1.441 \times 10^{-6}$ m^4



Cantilever beam with point load P.



Synthetic Dataset

- $N = 400$ samples: δ computed from the exact law + 2% Gaussian noise.
- Inputs sampled uniformly:
  - $P \sim \mathrm{Uniform}(500, 5000)$ N
  - $L \sim \mathrm{Uniform}(0.5, 2.0)$ m
- **70/30 train/test split**: $n_{\text{train}} = 280$, $n_{\text{test}} = 120$.
- Data scaled so values are $\sim\mathcal{O}(1)$:

$$p = \frac{P}{P_{\text{scale}}}, \quad l = \frac{L}{L_{\text{scale}}}, \quad \delta_{\text{scaled}} = \frac{\delta}{\delta_{\text{scale}}}$$

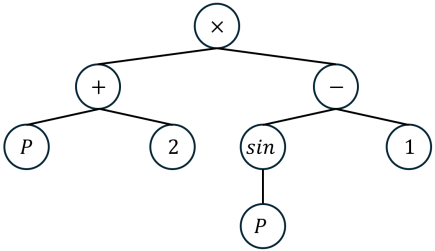
The SR algorithm receives only (P, L, δ) pairs — it does *not* know the governing equation.

Expression Trees

In tree-based GP, each candidate equation is an **expression tree**:

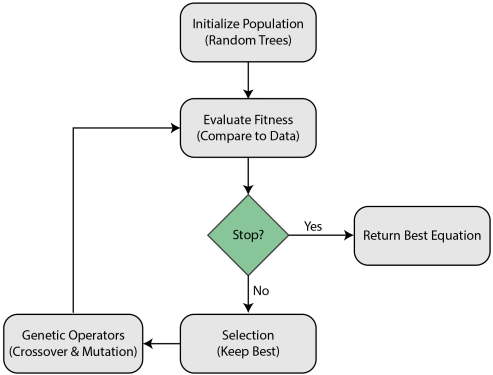
- **Internal nodes**: operators $(+, -, \times, \div, \sin, \ldots)$
- **Leaf nodes**: terminals (variables P, L or constants)

This representation enables genetic operators to manipulate the *structure* of expressions, not just their coefficients.



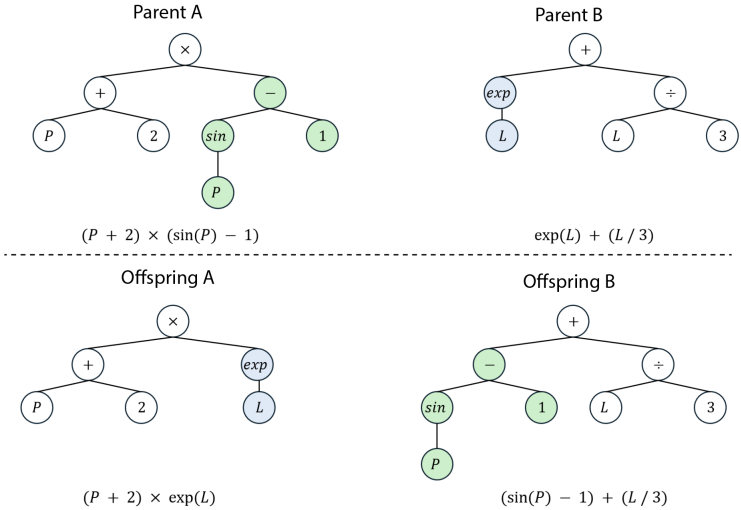
$$(P + 2) \times (\sin(P) - 1)$$

The Evolutionary Loop



1. **Initialize** random expression trees.
2. **Evaluate fitness** (e.g. MSE).
3. **Check stopping criterion.**
4. **Selection** (tournament: sample subset, pick best).
5. **Genetic operators** (crossover, mutation) $\rightarrow$ next generation.

Crossover (Recombination)



- Select a subtree in each parent and **swap** them.

Mutation

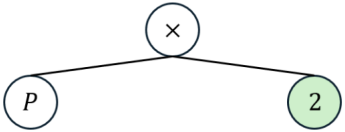
Introduces random variation to maintain diversity:

Point mutation: replace a single node.
Example: constant 2 $\rightarrow$ variable L, changing $P \times 2$ into $P \times L$.

Other types:

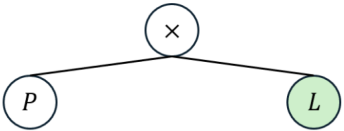
- **Subtree mutation:** replace a random subtree with a new one.
- **Hoist mutation:** replace a subtree with one of its own subtrees (reduces bloat).

Original



$P \times 2$

Mutated



$P \times L$

Fitness and Parsimony Pressure

Fitness function: quantifies how well a candidate expression matches training data (e.g. MSE).

Problem — Bloat: trees tend to grow in size without improving predictions.

Solution — Parsimony penalty:

$$\text{Fitness} = \text{MSE} + c \cdot N_{\text{nodes}}$$

where $c > 0$ is a user-chosen constant (here $c = 2 \times 10^{-4}$).

This aligns with **Occam's razor** from Chapter 2: among models with similar performance, prefer the simpler one.

Training Configuration

- Population size: 450
- Generations: 30
- Tournament size: 5
- Function set: $\{+, -, \times, \div\}$
- Parsimony coefficient: 2×10^{-4}

Genetic operator probabilities:

- Crossover: 0.7
- Subtree mutation: 0.1
- Hoist mutation: 0.05
- Point mutation: 0.1

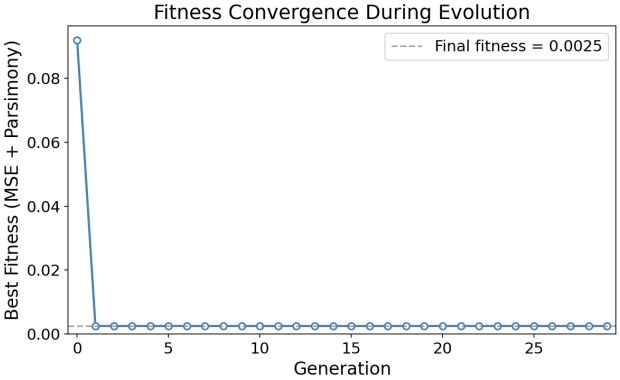
Linear scaling: For each candidate $g(\mathrm{x})$, fitness is evaluated after applying $\hat{y} = \alpha\, g(\mathrm{x}) + \beta$, letting GP focus on *structure* rather than scale.

Watching the Evolution

Gen	Fitness	Size	Discovered Form $g(p, l)$
0	0.0919	43	$c_1 l^2 p + c_2 l^2 + c_3 lp + c_4 p$
1	0.0025	7	$l^3 p$
2–29	0.0025	7	$l^3 p$ (unchanged)

- Gen 0: random initialization — poor fit (43 nodes).
- **Gen 1: correct structure discovered!** Compact 7-node expression.
- Gen 2–29: best fitness does not improve — correct form already found.

Fitness Convergence



Cumulative best fitness vs. generation. Sharp drop at Gen 1 when I^3p is discovered.

Discovered Equation

After unscaling to physical units, the SR-discovered law is:

$$\boxed{\delta \approx 1.10 \times 10^{-6} \cdot P\,L^3 + 2.82 \times 10^{-5}}$$

Comparison with Euler–Bernoulli theory:

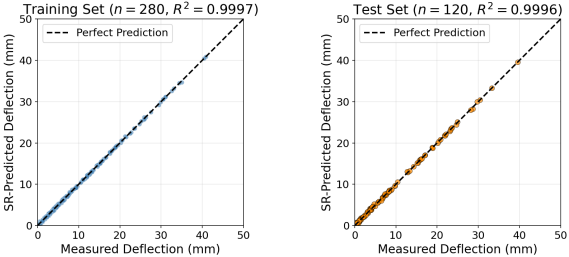
$$\frac{1}{3EI} = \frac{1}{3 \times 210 \times 10^9 \times 1.441 \times 10^{-6}} = 1.101 \times 10^{-6}$$

- SR coefficient 1.10×10^{-6} matches the theoretical value almost perfectly.
- The small additive constant 2.82×10^{-5} is negligible and can be dropped.
- GP has effectively **recovered the true physical law**.

Generalization: Training vs. Test

Metric	Train	Test
n	280	120
MSE	0.00110	0.00127
RMSE	0.0331	0.0356
R^2	0.9997	0.9996

Train and test errors are nearly identical $\Rightarrow$ **no overfitting**.



Parity plots: train (left), test (right).

Key Takeaways

1. **Symbolic regression** searches over equation *structure*, not just coefficients.
2. **Genetic programming** evolves expression trees through selection, crossover, and mutation.
3. **Parsimony pressure** combats bloat and promotes simple, interpretable expressions.
4. **Linear scaling** lets GP focus on structure by handling scale/offset externally.
5. SR is valuable when **no closed-form relationship is known** — discovered expressions serve as heuristics.
6. Always **validate on held-out test data** to check for overfitting.